

Session 7

Introduction To Packages

By : **Rajashekar G.S**
Java Full Stack Trainer

Review

- ❑ Whenever an error is encountered during run time, an exception occurs.
- ❑ A Java exception is an object that describes an exceptional condition that has occurred in a piece of code.
- ❑ When an exceptional condition arises, an object representing that exception is created and *thrown* in the method that caused the error.
- ❑ Java exception handling is managed using try, catch, throw, throws, and finally.
- ❑ Program statements to monitor are contained within a try block. Code within catch block catches the exception and handles it.
- ❑ Any code that absolutely must be executed before a method returns is put in a finally block.
- ❑ To manually throw an exception, use the keyword throw. Any exception that is thrown out of a method must be specified by a throws clause.

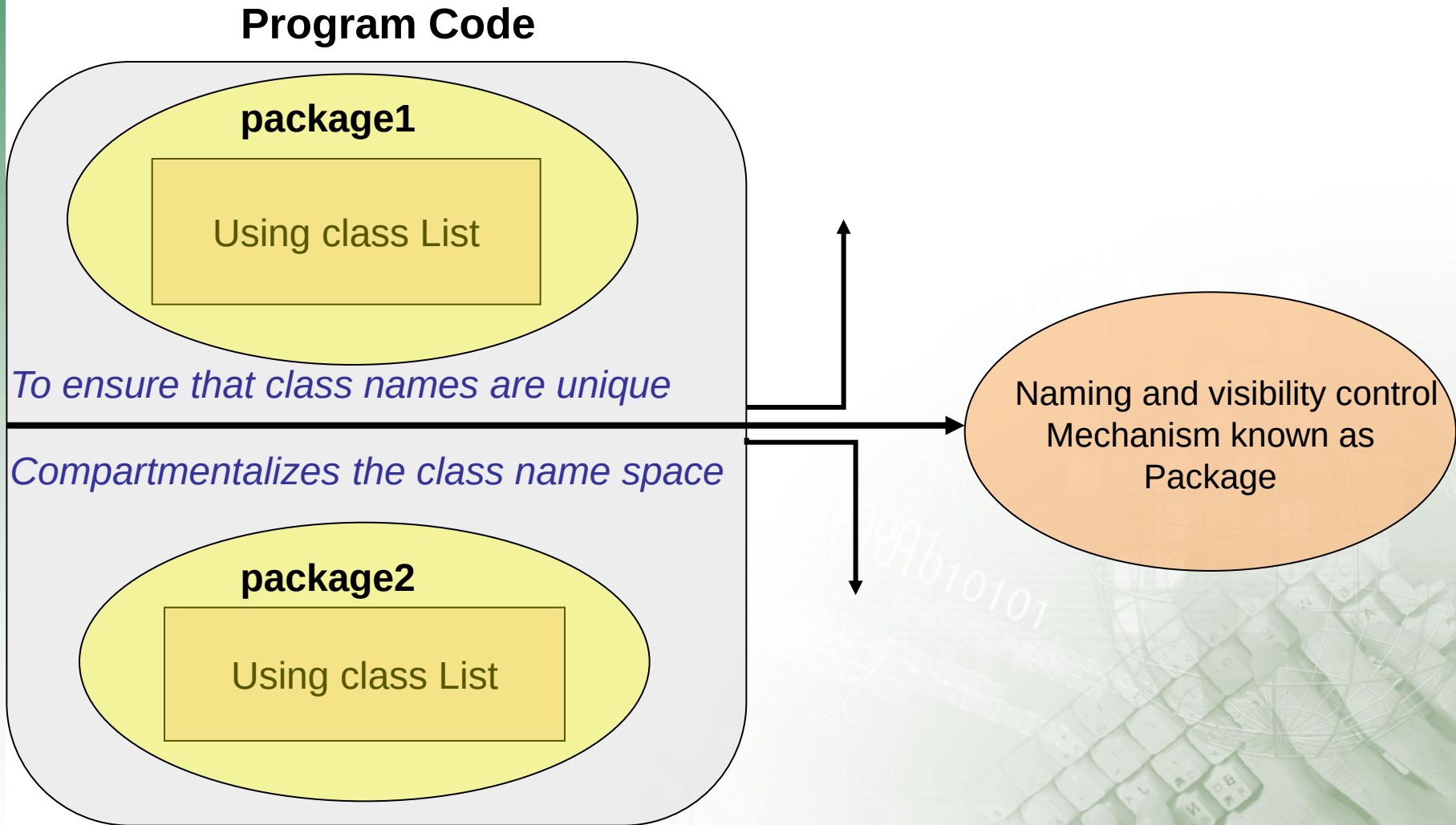
Objectives

- ❑ *Discuss the java.lang package*
- ❑ *Identify the various Wrapper classes*
- ❑ *Explain the String and StringBuffer classes*
- ❑ *Discuss the concept of immutability*
- ❑ *Identify the methods of the following classes:*
 - ❑ *Math*
 - ❑ *Runtime*
 - ❑ *System*
 - ❑ *Class*
 - ❑ *Object*

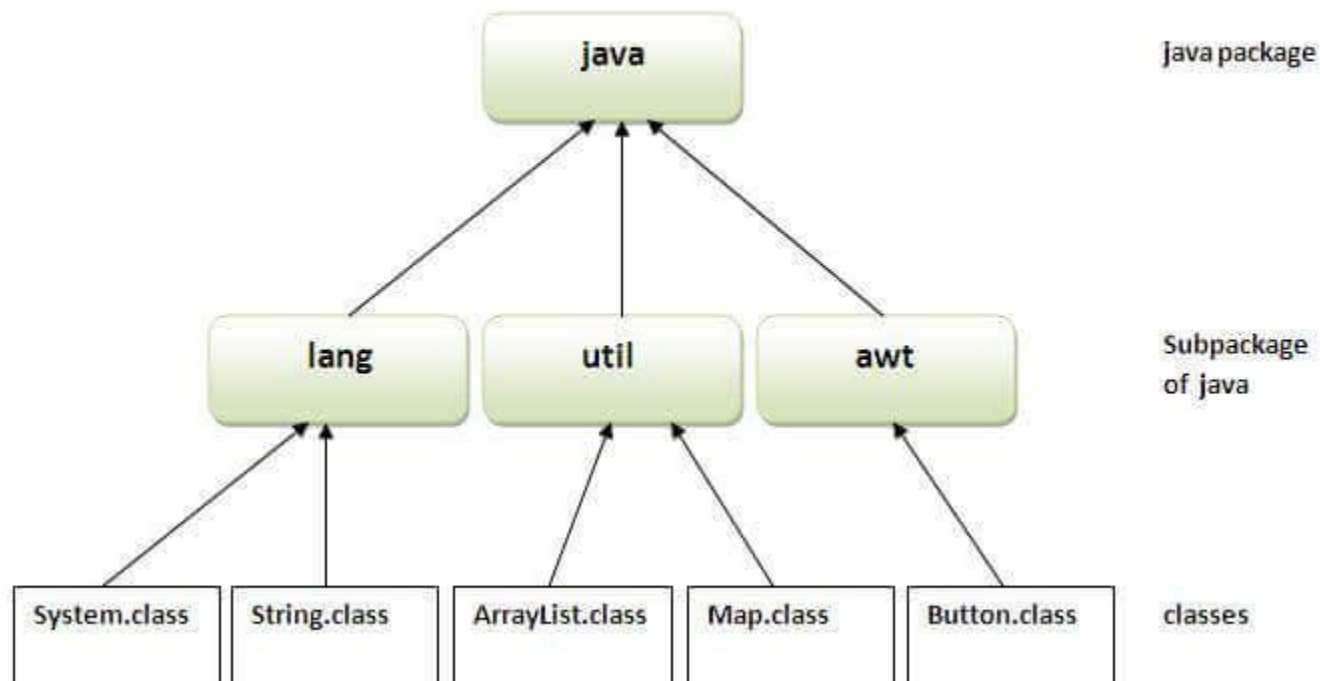
Java Package

- ❑ A java package is a group of similar types of classes, interfaces and sub-packages.
- ❑ Package in java can be categorized in two form, built-in package and user-defined package.
- ❑ There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.
- ❑ Here, we will have the detailed learning of creating and using user-defined packages.

Packages



Advantages of Java Package



Creating Packages in Java 5-1

Syntax

```
package <packagename>;
```

<packagename> is the valid identifier following package naming conventions.

```
...  
package pack;  
public class Welcome {  
    public void greet() {  
        System.out.println("hello");  
    }  
}  
...
```

Creating Packages in Java 5-2

Syntax

```
import <package-name>.*;  
import <package-name>.<class-name>;
```

It is possible to include the same package statement in more than one Java source file to indicate that all such classes belong to the same package.

```
import pack.Welcome;  
class Importer {  
    public static void main(final String [] args) {  
        Welcome.greet();  
    }  
}
```

Package naming convention

- ❑ Package names must be a group of words starting with all lowercase domain names (e.g. com, org, net, etc).
- ❑ Subsequent parts of the package name may be different according to an organization's own internal naming conventions.
- ❑ We should follow reverse model of domain names.

```
package com.rajtech.training.mybank.controller;  
package com.company.myapplication.web.controller;  
package com.google.search.common;
```

Creating Packages in Java 5-3

```
package sun;  
/** This class demonstrates uses of Packages.  
 */  
public class Softwares {  
    /** Constructor. */  
    public Softwares() {  
    }  
    /** This method contains a list of softwares. */  
    public final void listSoftware() {  
        int i;  
        String []arrSun = {"JCreator", "JBuilder", "Jacobe"};  
        for (i = 0; i < 3; i++) {  
            System.out.println(arrSun[i]);  
        }  
    }  
}
```

Demonstration: Example 1

Creating Packages in Java 5-4

❑ *Creating directories that contain the package and bear*

```
/** Creates a package oracle.*/  
package oracle;  
public class Softwares {  
    /** Constructor. */  
    public Softwares() {  
    }  
    /** This method contains a list of softwares. */  
    public final void printSoftware() {  
        int i;  
        String []arrOracle = {"Oracle8i" , "Oracle9i" ,  
"Oracle10G"};  
        for (i = 0; i < 3; i++) {  
            System.out.println(arrOracle[i]);  
        }  
    }  
}
```

Creating Packages in Java 5-5

```
class SoftwareList {  
    /** Constructor. */  
    protected SoftwareList()  
    {  
    }  
    /** This is a main method  
     * It creates objects of  
and access its methods  
     * @param args passed to the main method  
     */  
    public static void main(final String []args) {  
        sun.Softwares objSun = new sun.Softwares();  
        objSun.listSoftware();  
        oracle.Softwares objOracle = new oracle.Softwares();  
        objOracle.printSoftware();  
    }  
}
```

Current working directory (Can include SoftwareList.java here)

sun directory (contains Softwares.java)

oracle directory (contains Softwares.java)

How to compile java package

- ❑ If you are not using any IDE, you need to follow the syntax given below in command prompt:

```
javac -d directory javafilename
```

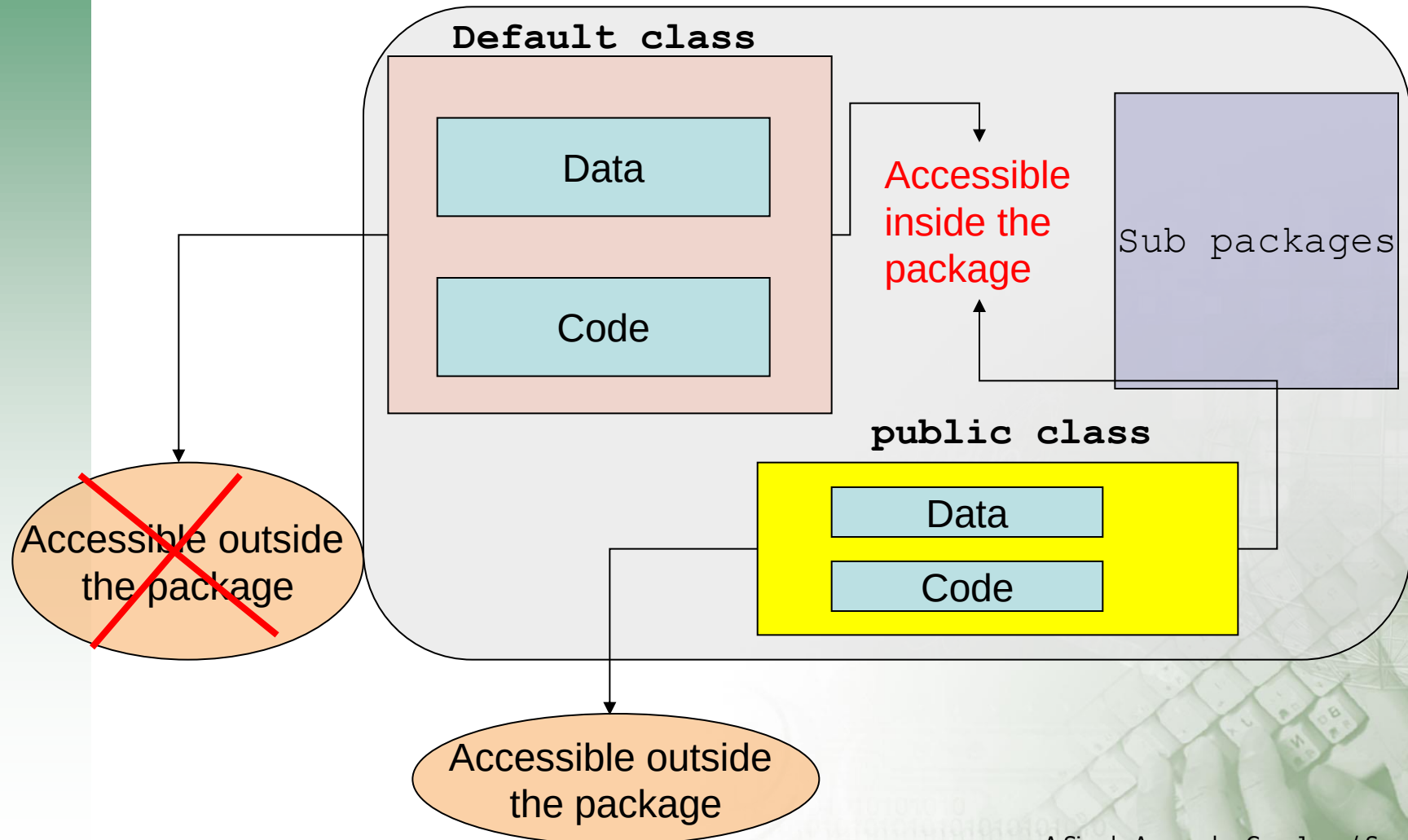
```
javac -d . Simple.java
```

To Compile: `javac -d . Simple.java`
To Run: `java mypack.Simple`

- The -d switch specifies the destination where to put the generated class file.
- You can use any directory name like /home (in case of Linux), d:/abc (in case of windows) etc.
- If you want to keep the package within the same directory, you can use . (dot).
- mypack is the package name were Simple.java is coded.

Packages and Access control 3-1

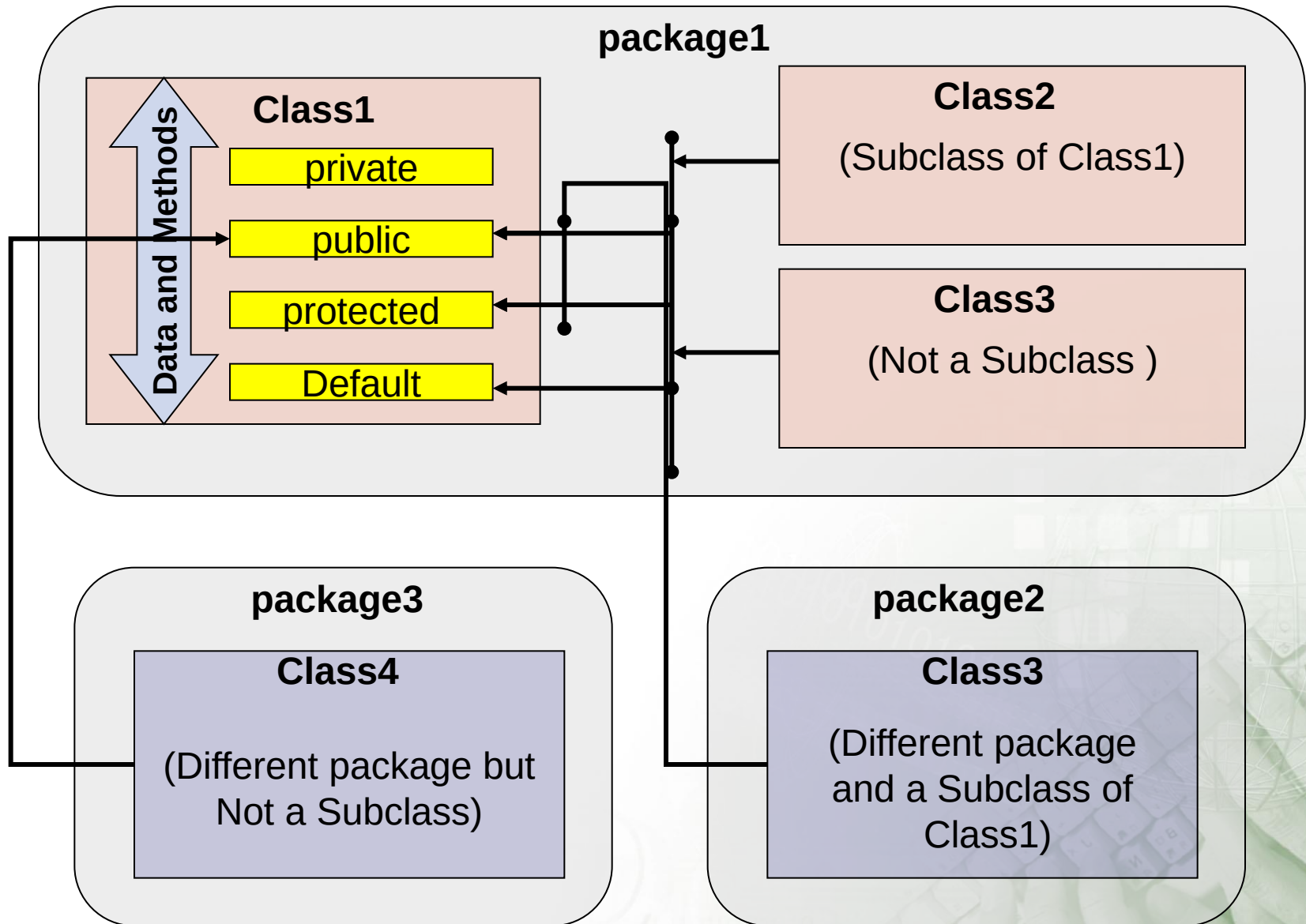
package (Container for classes and sub packages)



Packages and Access control 3-2

Location	private	No modifier	protected	public
Same class	Yes	Yes	Yes	Yes
Same package and also a subclass	No	Yes	Yes	Yes
Same package but not a subclass	No	Yes	Yes	Yes
Different package but a subclass	No	No	Yes	Yes
Different package but not a subclass	No	No	No	Yes

Packages and Access control 3-3



Wrapper Classes

- ❑ A Wrapper class is a class whose object wraps or contains a primitive data types.
- ❑ When we create an object to a wrapper class, it contains a field and in this field, we can store a primitive data types.
- ❑ In other words, we can wrap a primitive value into a wrapper class object.

Why Wrapper Classes ?

- ❑ They convert primitive data types into objects. Objects are needed if we wish to modify the arguments passed into a method (because primitive types are passed by value).
- ❑ The classes in `java.util` package handles only objects and hence wrapper classes help in this case also.
- ❑ Data structures in the Collection framework, such as `ArrayList` and `Vector`, store only objects (reference types) and not primitive types.
- ❑ An object is needed to support synchronization in multithreading.
- ❑ Wrapper class objects allow null values while primitive data type doesn't allow it.

Wrapper class 5-1

- Wrapper classes encapsulate simple primitive types in the form of classes.

```
int digit = 10;
```

```
...
```

```
int num1 = 5;
```

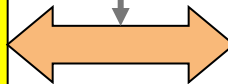
```
Integer num = new Integer(num1);
```

```
int num2 = num.intValue();
```

```
...
```

Java.lang provides

**Primitive
data type**



**Wrapper
class**

Wrapper class 5-2

Primitive Data Types	Wrapper Classes
byte	Byte
char	Character
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
short	Short

Wrapper class 5-3

```
class NumberWrap {
    /** Constructor.*/
    protected NumberWrap() {
    }
    /** This is a main method.
     * It converts the primitive values to its wrapper
     counterparts
     * @param args passed to the main method
     */
    public static void main(final String[] args) {
        String number = args[0];
        Byte byNum = Byte.valueOf(number);
        Short shNum = Short.valueOf(number);
        Integer num = Integer.valueOf(number);
        Long lgNum = Long.valueOf(number);
        System.out.println("Output");
        System.out.println(byNum);
        System.out.println(shNum);
        System.out.println(num);
        System.out.println(lgNum);
    }
}
```

Wrapper class 5-4

Method	Description
isDigit()	Determines if the character is a numeric digit between 0-9.
isLetter()	Determines if the character is a letter.
isLowerCase()	Determines if the character is in lowercase.
isUpperCase()	Determines if the character is in uppercase.
isSpace()	Determines if the character is a space or a return.
isUnicodeIdentifier()	Determines if it is a single field with type char.

Wrapper class 5-5

```
class TestCharacter {
    /** Constructor.*/
    protected TestCharacter() {
    }
    public static void main(final String[] args) {
        int count;
        char[] values = {'*', '7', 'p', ' ', 'P'};
        for (count = 0; count < values.length; count++) {
            if (Character.isDigit(values[count])) {
                System.out.println(values[count] + " is a digit");
            } if (Character.isLetter(values[count])) {
                System.out.println(values[count] + " is a letter");
            } if (Character.isUpperCase(values[count])) {
                System.out.println(values[count] + " is uppercase");
            } if (Character.isUnicodeIdentifierStart(values[count])) {
                System.out.println(values[count] + " is valid first"
                    + " character of Unicode identifier");
            }
        }
    }
}
```

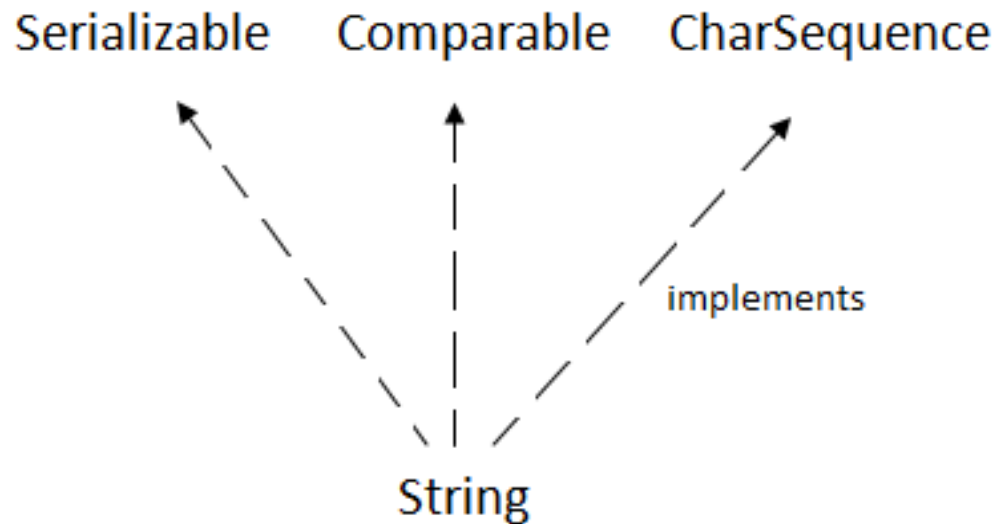
String Class

- ❑ **String** is a sequence of characters, for e.g. “ROSE” is a string of 4 characters.
- ❑ In java, string is an immutable object which means it is constant and cannot be changed once it has been created.



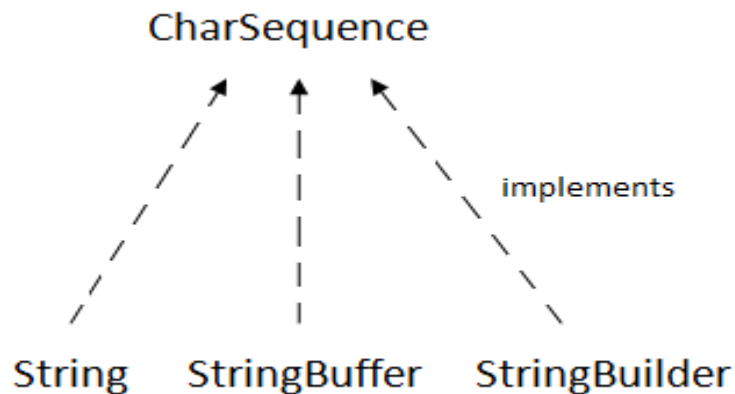
String Class

- ❑ The `java.lang.String` class implements *Serializable*, *Comparable* and *CharSequence* interfaces.



CharSequence Interface

- ❑ The CharSequence interface is used to represent the sequence of characters.
- ❑ String, StringBuffer and StringBuilder classes implement it.
- ❑ It means, we can create strings in java by using these three classes.



- **The Java String is immutable which means it cannot be changed.**
- **Whenever we change any string, a new instance is created.**
- **For mutable strings, you can use StringBuffer and StringBuilder classes.**

Creating a String

There are two ways to create a String in Java

- ❑ String literal
- ❑ Using new keyword

String Literal

- ❑ Java String literal is created by using double quotes. For Example:

`String s="welcome";`

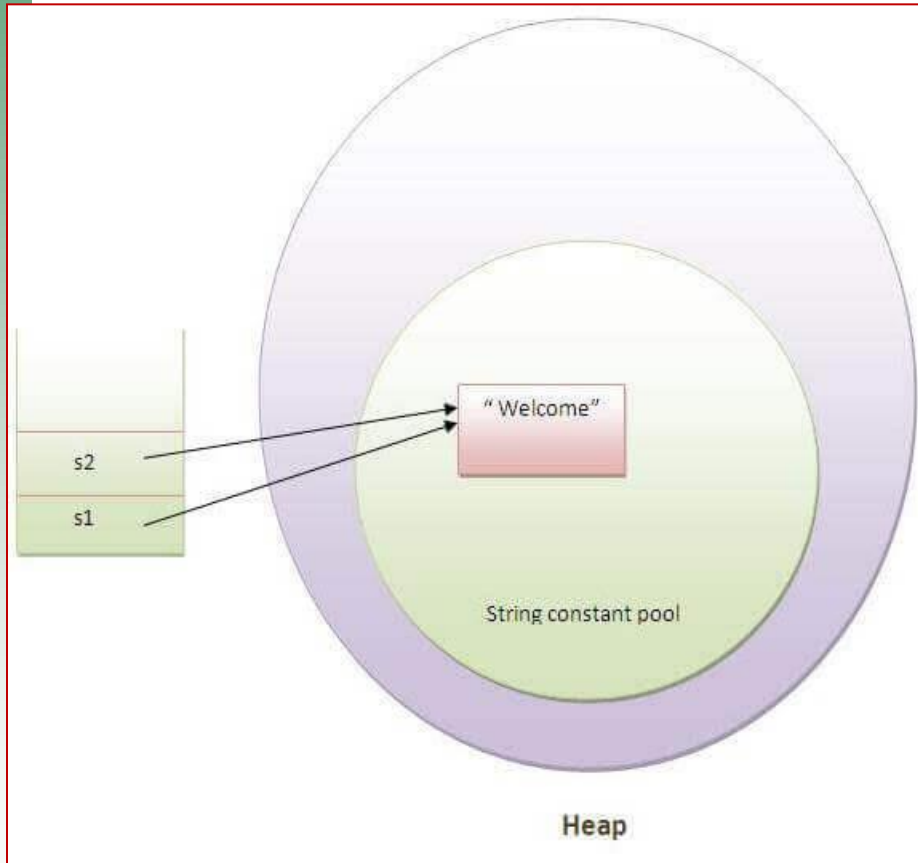
- ❑ Each time you create a string literal, the JVM checks the "string constant pool" first.
- ❑ If the string already exists in the pool, a reference to the pooled instance is returned.
- ❑ If the string doesn't exist in the pool, a new string instance is created and placed in the pool.

For example:

- ❑ `String s1="Welcome";`
- ❑ `String s2="Welcome";`//It doesn't create a new instance

String Literal

- ❑ In this example, only one object will be created.
- ❑ JVM will not find any string object with the value "Welcome" in string constant pool, that is why it will create a new object.
- ❑ Once it will find the string with the value "Welcome" in the pool, it will not create a new object but will return the reference to the same instance.
- ❑ To make Java more memory efficient (because no new objects are created if it exists already in the string constant pool).

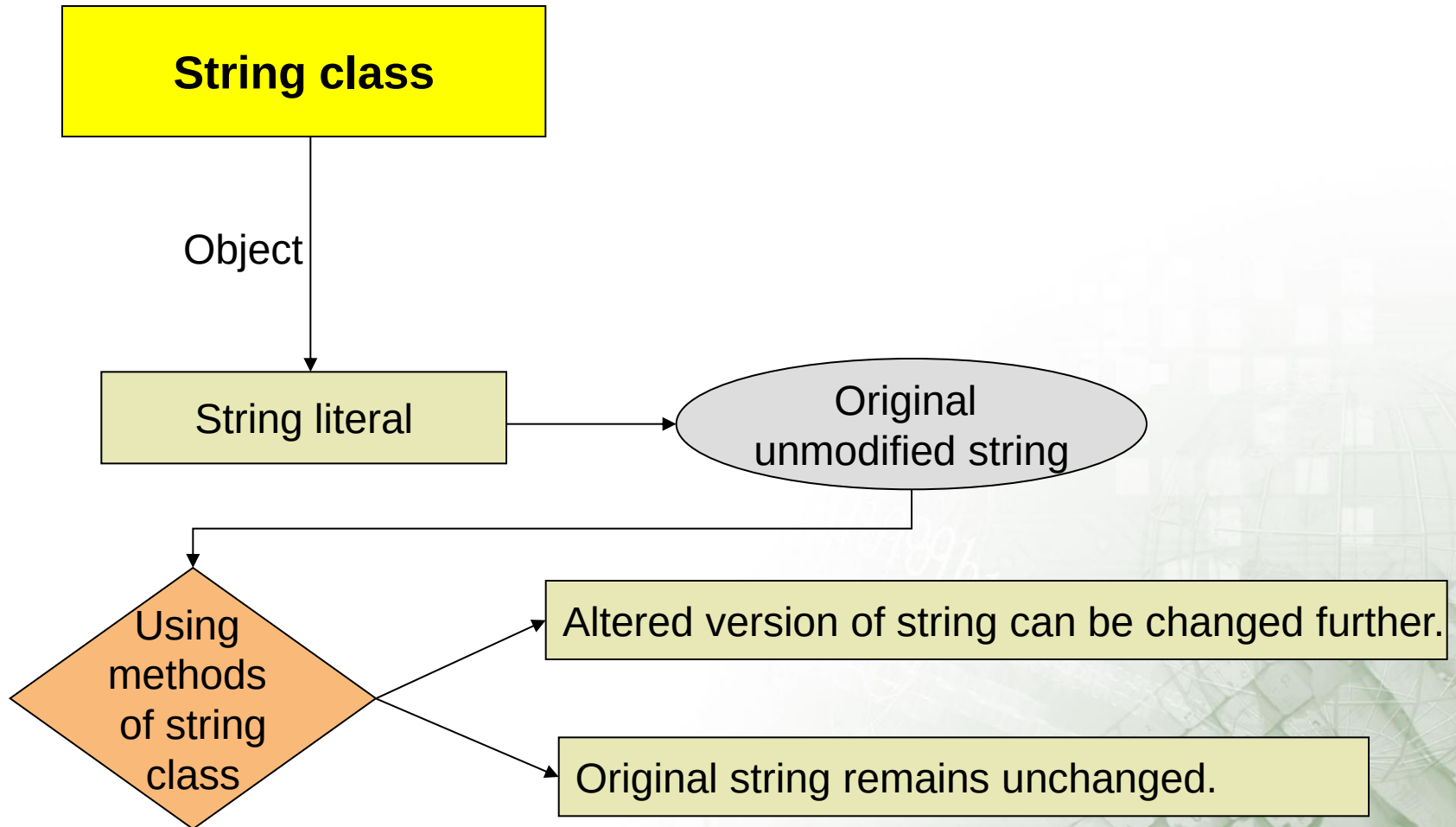


By new keyword

String s=new String("Welcome");//creates two objects and one reference variable

- ❑ In such case, JVM will create a new string object in normal (non-pool) heap memory, and the literal "Welcome" will be placed in the string constant pool.
- ❑ The variable s will refer to the object in a heap (non-pool).

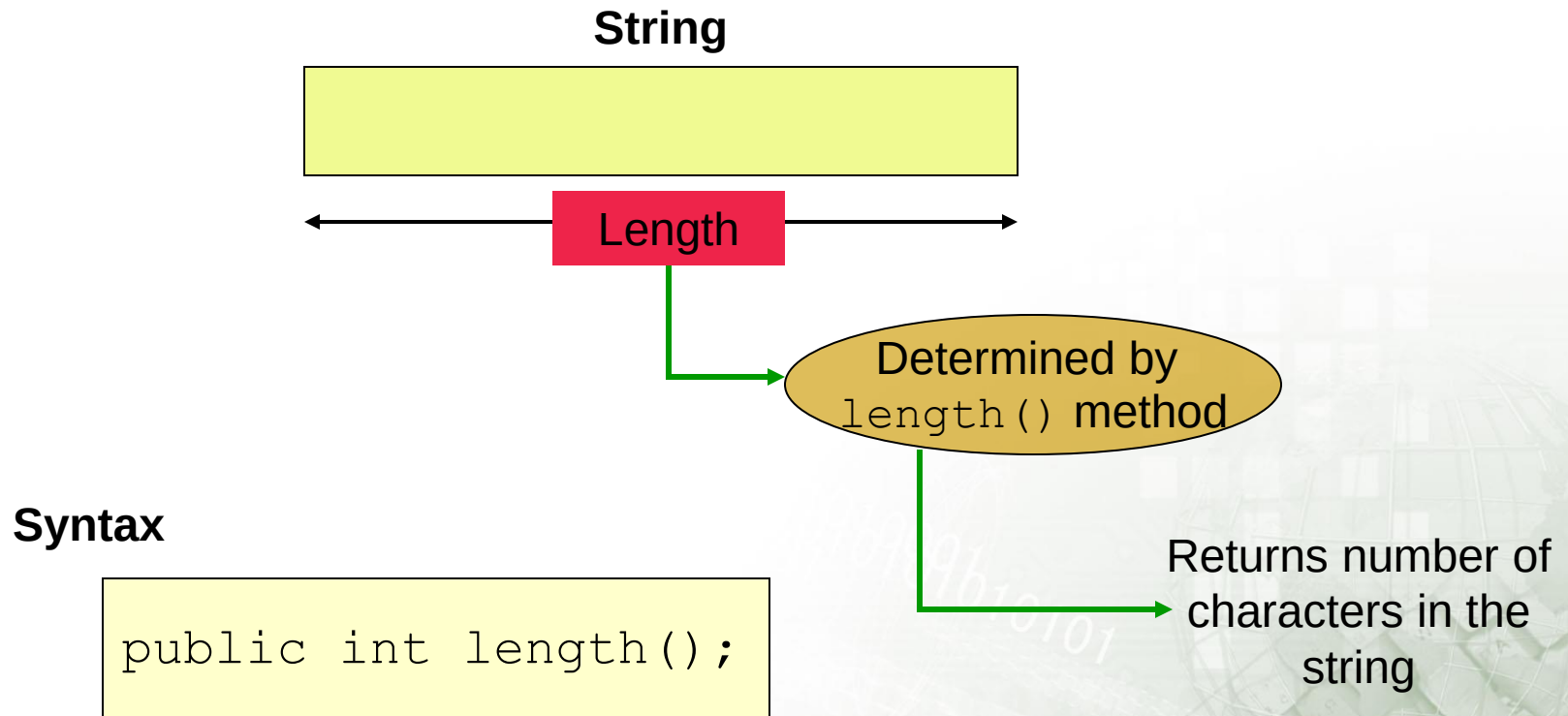
String class



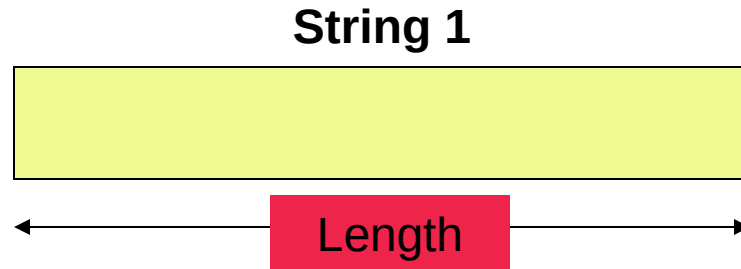
Constructors of the String class

Constructor	Description
String()	This creates an empty string.
String(String value)	This creates a new string that is a copy of the given string.
String (char [] value)	This constructs a new string based on the character array.
String(byte [] value)	This creates a new String by converting the given array of bytes.

String length 2-1

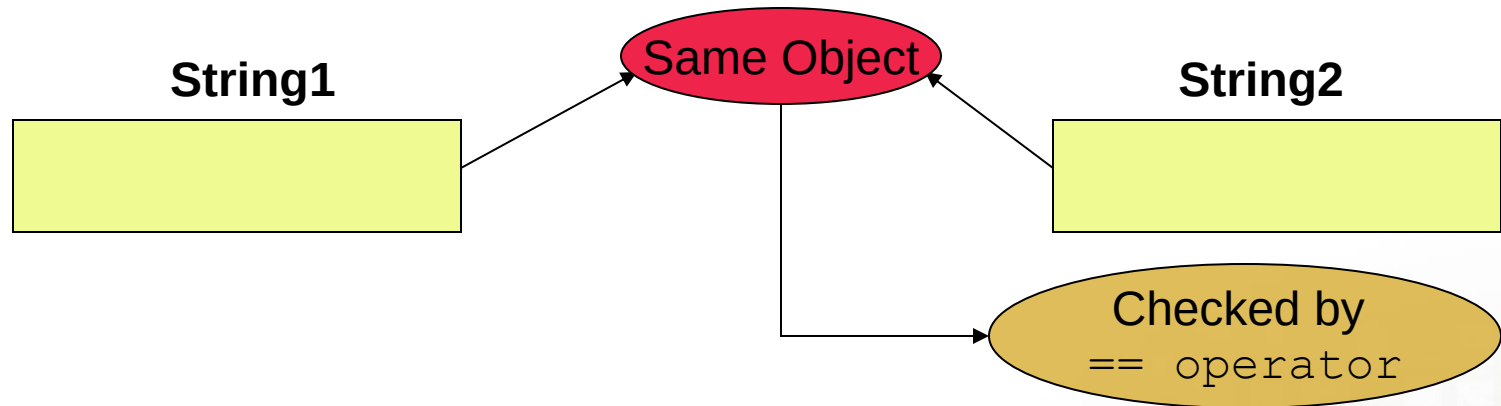


String length 2-2

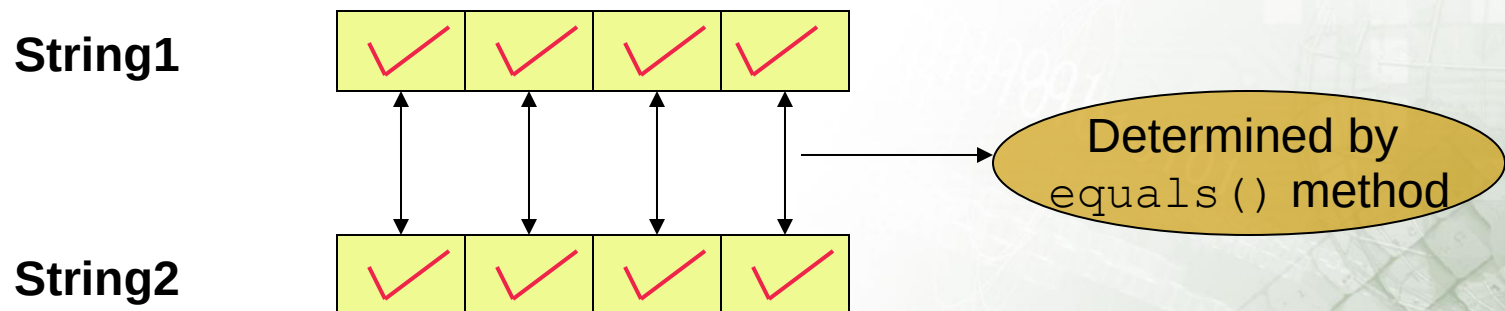


```
String name = "John Smith";  
System.out.println (name.length());
```

String comparison 4-1



Checks whether the strings refer to same or different objects



Checks the characters forming the contents of the strings

String comparison 4-2

```
if (string1 == string2) {  
    System.out.println("String 1 and String 2 are equal");  
} else {  
    System.out.println("String 1 and String 2 are not equal");  
} if (string1.equals(string3)) {  
    System.out.println("String 1 and String 3 are equal");  
} else {  
    System.out.println("String 1 and String 3 are not  
equal");  
} System.out.println("Setting string1 equal to string2");  
string2 = string1;  
if (string1.equals(string2)) {  
    System.out.println("Both Strings are equal");  
} else {  
    System.out.println("Both Strings are not equal");  
}  
}
```

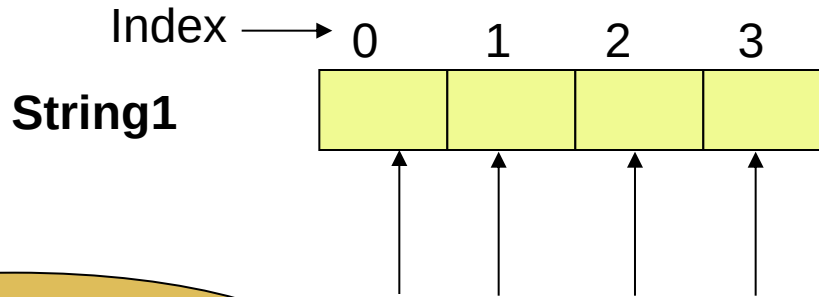
String comparison 4-3

Method	Description
boolean equalsIgnoreCase (String value)	This method compares two strings ignoring their case.
int compareTo(String value)	Compares two strings alphabetically. It returns 0 if the two strings are equal. The result is a negative integer if the String object lexicographically precedes the argument string, or a positive integer if the String object lexicographically follows the argument string.
boolean startsWith(String value)	To find if a string starts with another string.
boolean endsWith(String value)	To find if a string ends with another string.

String comparison 4-4

```
if (string1 == string2) {  
    System.out.println("String A and B refer to same object");  
} else {  
    System.out.println("String A and B refer to different object");  
} if (string1.equals(string2)) {  
    System.out.println("String A and B have same content");  
    if (string1.endsWith("r")) {  
        System.out.println("Ends with r");  
    }  
}  
  
} else if (string1.equalsIgnoreCase(string3)) {  
    System.out.println("String A and C have same content");  
} if (string1.compareTo("Answer") == 0) {  
    System.out.println("String A have same content alphabetically  
as Answer");  
} if (string1.startsWith("A")) {  
    System.out.println("Starts with A");  
  
    System.out.println("String C is " + string3);  
}
```

Searching Strings 2-1



Method indexOf(character)



First match found

Case 1: Returns location index where first match found

Case 2: Returns -1 if no match found

Searching Strings 2-2

```
class SearchString {
    /** Constructor. */
    protected SearchString() {
    }
    /** This is a main method.
     * It demonstrates searching within string
     * @param args passed to the main method
     */
    public static void main(final String []args) {
        String name = "JohnSmith@123.com";
        System.out.println("Email-id is: " + name);
        System.out.println("Index of @ is:");
        System.out.println("Index of . is:");
        System.out.println(name.indexOf('.'));
        if (name.indexOf('.') > name.indexOf('@')) {
            System.out.println("The E-mail id is valid");
        } else {
            System.out.println("The E-mail id is not valid");
        }
    }
}
```

Extracting Strings 3-1

Method	Description
<code>public char charAt(int index)</code>	This is used to extract a single character from given position specified with index and the value mentioned at index should be nonnegative
<code>public String substring(int index)</code>	This is used to extract portions of a string starting from position index.
<code>public String substring(int beginindex, int endindex)</code>	This is used to extract portions of a string starting from position beginindex upto endindex
<code>public String concat(String str)</code>	This is used to concatenate two strings and create a new String object that contains the invoking string
<code>public String replace(char old, char new)</code>	This is used to replace all occurrences of one character in the invoking string with another character
<code>public String trim()</code>	This method is used to return a copy of the invoking string without any leading and trailing whitespace

Extracting Strings 3-2

Method	Description
<code>public char charAt(int index)</code>	This is used to extract a single character from given position specified with index and the value mentioned at index should be nonnegative.
public String ... public String char ch; public String endindex, ch = "orange".charAt(3); public String ...	f a string f a string upto strings and contains the
<code>public String replace(char old, char new)</code>	This is used to replace all occurrences of one character in the invoking string with character.
public String This will extract the single character "n" from index(3) and store it inside variable ch.	method is used to return a copy of the g string without any leading and whitespace.

Extracting Strings 3-3

```
class StringMethods {  
    /** Constructor. */  
    protected StringMethods() {  
    }  
    /** This is a main method.  
     * @param args passed to the main method  
     */  
    public static void main(final String [] args) {
```

```
        System.out.println(s3.toLowerCase());  
        System.out.println(s1.trim());  
    }}
```

```
        System.out.println(s);  
        System.out.println("index of t = " + s.indexOf('t'));  
        System.out.println("last index of t = " + s.lastIndexOf('t'));  
        System.out.println("index of(t, 10) = " + s.indexOf('t', 10));  
        System.out.println("last index of(t, 60) = " +  
        s.lastIndexOf('t', 60));  
        System.out.println(s1.substring(6));  
        System.out.println(s1.substring(3, 8));  
        System.out.println(s2.concat("World"));  
        System.out.println(s2.replace('l', 'w'));
```

Changing case of character within a string 2-1



Syntax

```
public String toUpperCase();
```



Syntax

```
Public String toLowerCase();
```

Changing case of character within a string 2-2

```
class StringTest {
    /** Constructor. */
    protected StringTest() {
    }
    /** This is a main method.
     * It demonstrates the length() and UpperCase()
methods of Strings
     * @param args passed to the main method
     */
    public static void main(final String [] args) {
        String name = new String("George");
        System.out.println("Name is " + name);
        int length = name.length();
        System.out.println("Length of name " + length + "
characters");
        System.out.println("Name in UpperCase will be: ");
        String nameUppercase = name.toUpperCase();
        System.out.println(nameUppercase);
    }
}
```

StringBuffer class 2-1

- ❑ StringBuffer represents a string that can be modified.
- ❑ Strings that use concatenation operator(+) create a string buffer object automatically.

Constructors	Description
<code>public StringBuffer()</code>	Reserves room for 16 characters.
<code>public StringBuffer (int length)</code>	This sets the size of the buffer.
<code>public StringBuffer(Stringvalue)</code>	Accepts a String as an argument that sets the initial content and reserves room for 16 more characters without reallocation.

StringBuffer class 2-2

Method	Description
StringBuffer insert(String s)	Inserts a string representation of boolean value at given position.
int length()	Determines length of StringBuffer object.
void setCharAt(int pos, char ch)	Sets the character at the position specified by pos with the new value specified by ch.
String toString()	Converts to String form.
StringBuffer reverse()	Reverses the characters within a StringBuffer object.
StringBuffer delete(int start, int end)	This will delete a sequence of characters from the invoking object starting from position specified by start till the index – 1 position specified by end.
StringBuffer deleteCharAt(int pos)	This will delete the character at the index specified by pos.
StringBuffer replace(int start, int end, String s)	It replaces one set of characters with another set. The replacement string s will replace from the position specified by start till the position specified by end.

Immutability

- ❑ An object is considered immutable if its state cannot change after it is constructed.
- ❑ Simply an immutable class is a class whose properties can not be modified after creation.
- ❑ That means at creation, all of the instance data is provided and remain unchanged till the destruction of the objects.
- ❑ Immutable classes are great for concurrent applications.
- ❑ As per their immutable nature, they help to maintain the application in non-corrupted and consistent behavior as the state of the instances can not be changed.

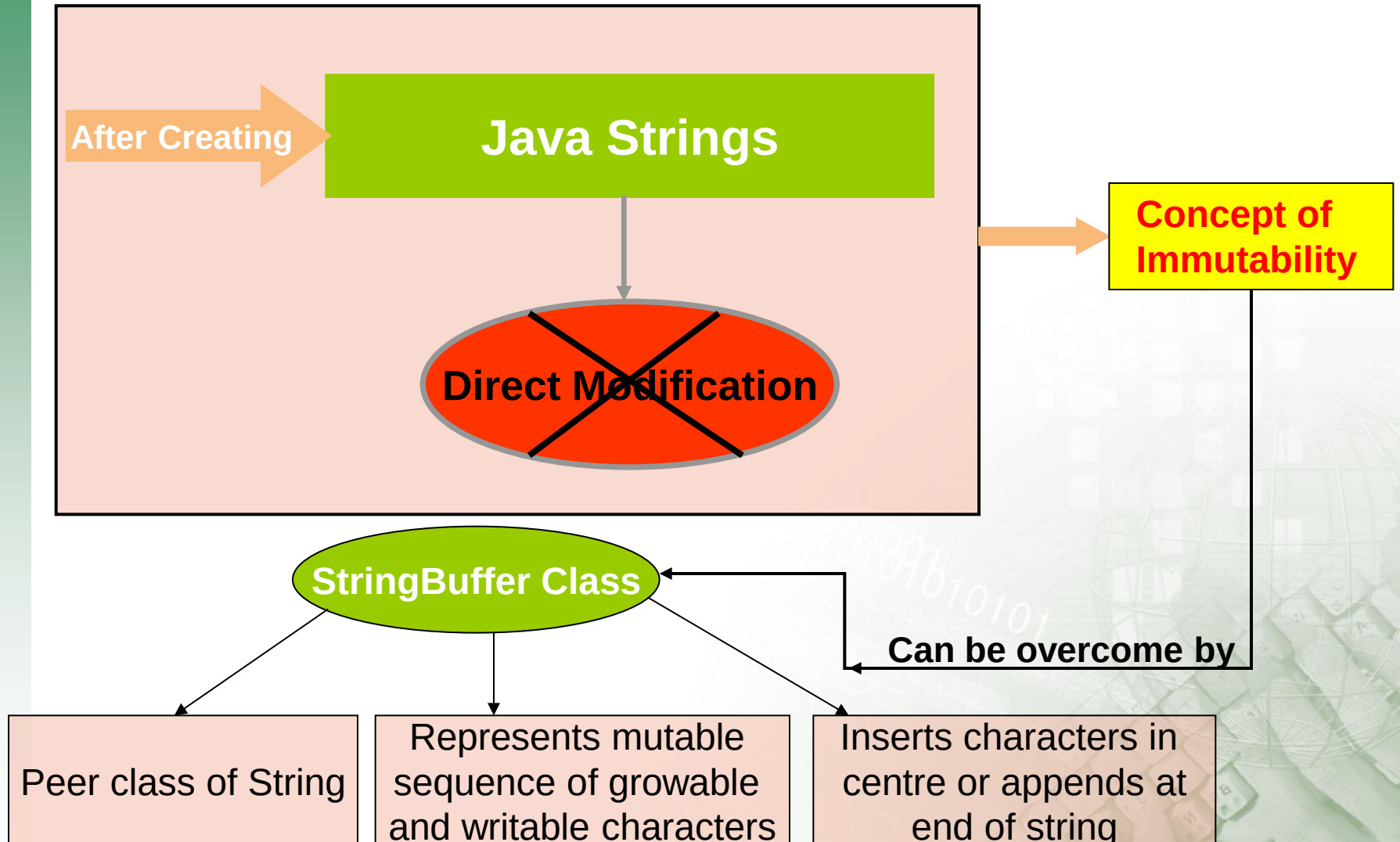
Rules for Writing Immutable Classes

- ❑ All fields are private & final
- ❑ Class can not be extended
- ❑ No public method can change the state

Examples

- ❑ String
- ❑ Boxed primitives like Integer, Long and etc.
- ❑ BigInteger
- ❑ BigDecimal.

Immutability 2-1



Immutability 2-2

```
class StringBuffer {  
    /** Constructor. */  
    protected StringBuffer() {  
    }  
    public static void main(final String []args) {  
        StringBuffer buf = new StringBuffer("Java");  
        buf.append(" Guide Ver1/"); // append // Java Guide Ver1/7  
        buf.append(7);  
        int index = 5; // Insert // Java Student Guide Ver1/7  
        buf.insert(index, "Student ");  
        index = 23; // Set // Java Student Guide Ver1.7  
        buf.setCharAt(index, '.');  
        int start = 24; // Replace // Java Student Guide Ver1.8  
        int end = 25;  
        buf.replace(start, end, "8");  
        String s = buf.toString(); //Convert to string  
        System.out.println(s);  
    }  
}
```

StringBuilder

- ❑ StringBuilder class is mutable sequence of characters.
- ❑ Java **StringBuilder** is identical to **StringBuffer** except for one important difference it is not **synchronized** , that means it is not thread safe.
- ❑ The main difference between StringBuffer and StringBuilder is StringBuffer is synchronized, but StringBuilder is not.
- ❑ Use StringBuilder in all cases where you have only a **single thread** accessing your object.

StringBuilder

- ❑ If execution speed and performance is a factor, StringBuilder class can be used in place of StringBuffer.
- ❑ StringBuilder Class has common the append() and insert() methods.
- ❑ These methods are overloaded within StringBuilder in order to accommodate different data type.

Java StringBuilder Constructors

CONSTRUCTOR	DESCRIPTION
<code>StringBuilder()</code>	Creates an empty string builder with a default capacity of 16 (16 empty elements).
<code>StringBuilder(CharSequence cs)</code>	Constructs a string builder containing the same characters as the specified <code>CharSequence</code> , plus an extra 16 empty elements trailing the <code>CharSequence</code> .
<code>StringBuilder(int initCapacity)</code>	Creates an empty string builder with the specified initial capacity.
<code>StringBuilder(String s)</code>	Creates a string builder whose value is initialized by the specified string, plus an extra 16 empty elements trailing the string.

StringBuilder Example

```
// creates empty builder, capacity 16
StringBuilder sb = new StringBuilder();
// adds 5 character string at beginning
sb.append("Hello");
System.out.println("StringBuilder length =
"+sb.length()); // prints 5
System.out.println("StringBuilder capacity
= "+sb.capacity()); // prints 16

sb.append("World");// now original string
is changed
System.out.println(sb); // prints Hello
World
```

StringBuilder vs StringBuffer

STRINGBUFFER	STRINGBUILDER
Synchronized, hence thread safe.	Not synchronized, not thread safe.
Operates slower due to thread safety feature	Better performance compared to StringBuffer
Has some extra methods – substring, length, capacity etc.	Not needed because these methods are present in String too.
Introduced in Java 1.2	Introduced in Java 1.5 for better performance.

When to use which one ?

- ❑ If a string is going to remain constant throughout the program, then use **String** class object because a String object is immutable.
- ❑ If a string can change (example: lots of logic and operations in the construction of the string) and will only be accessed from a single thread, using a **StringBuilder** is good enough.
- ❑ If a string can change, and will be accessed from multiple threads, use a StringBuffer because **StringBuffer** is synchronous so you have thread-safety.

StringTokenizer

- ❑ StringTokenizer class in Java is used to break a string into tokens.
- ❑ The `java.util.StringTokenizer` class allows an application to break a string into parts.
- ❑ Its methods do not distinguish among identifiers, numbers, and quoted strings.
- ❑ This class methods do not even recognize and skip comments.
- ❑ Introduced since `jdk1.0` .

Constructors

StringTokenizer(String str)

This constructor constructs a string tokenizer for the specified string.

StringTokenizer(String str, String delim)

This constructor constructs a string tokenizer for the specified string.

StringTokenizer(String str, String delim, boolean returnDelims)

This constructor constructs a string tokenizer for the specified string.

Methods

Modifier and Type	Method and Description
int	<u>countTokens()</u> Calculates the number of times that this tokenizer's nextToken method can be called before it generates an exception.
boolean	<u>hasMoreElements()</u> Returns the same value as the hasMoreTokens method.
boolean	<u>hasMoreTokens()</u> Tests if there are more tokens available from this tokenizer's string.
<u>Object</u>	<u>nextElement()</u> Returns the same value as the nextToken method, except that its declared return value is Object rather than String.
<u>String</u>	<u>nextToken()</u> Returns the next token from this string tokenizer.
<u>String</u>	<u>nextToken(String delim)</u> Returns the next token in this string tokenizer's string.

Example of the use of the tokenizer

```
StringTokenizer st = new StringTokenizer("this is  
a test");  
while (st.hasMoreTokens()) {  
    System.out.println(st.nextToken());  
}
```



```
this  
is  
a  
test
```

Delimiters In Tokens

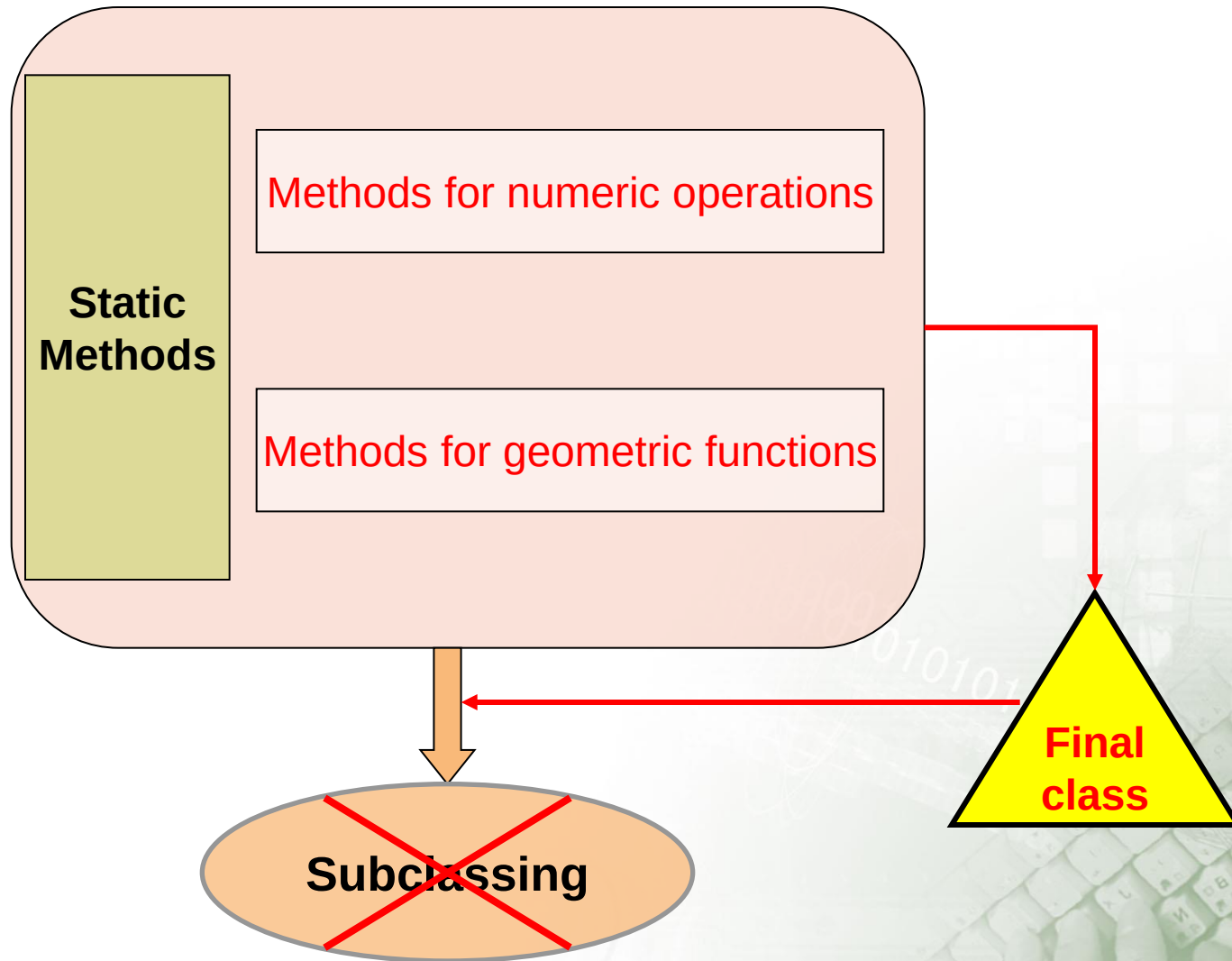
```
import java.util.StringTokenizer;

public class TokensWithDelimiter {
    public static void main(String a[]){
        String msg = "http://10.123.43.67:80/";
        StringTokenizer st = new
StringTokenizer(msg, "://.", true);
        while(st.hasMoreTokens()){
            System.out.println(st.nextToken());
        }
    }
}
```

http
:
/
/
10
.
123
.
43
.
67
:
80
/

Math class 3-1

Math class



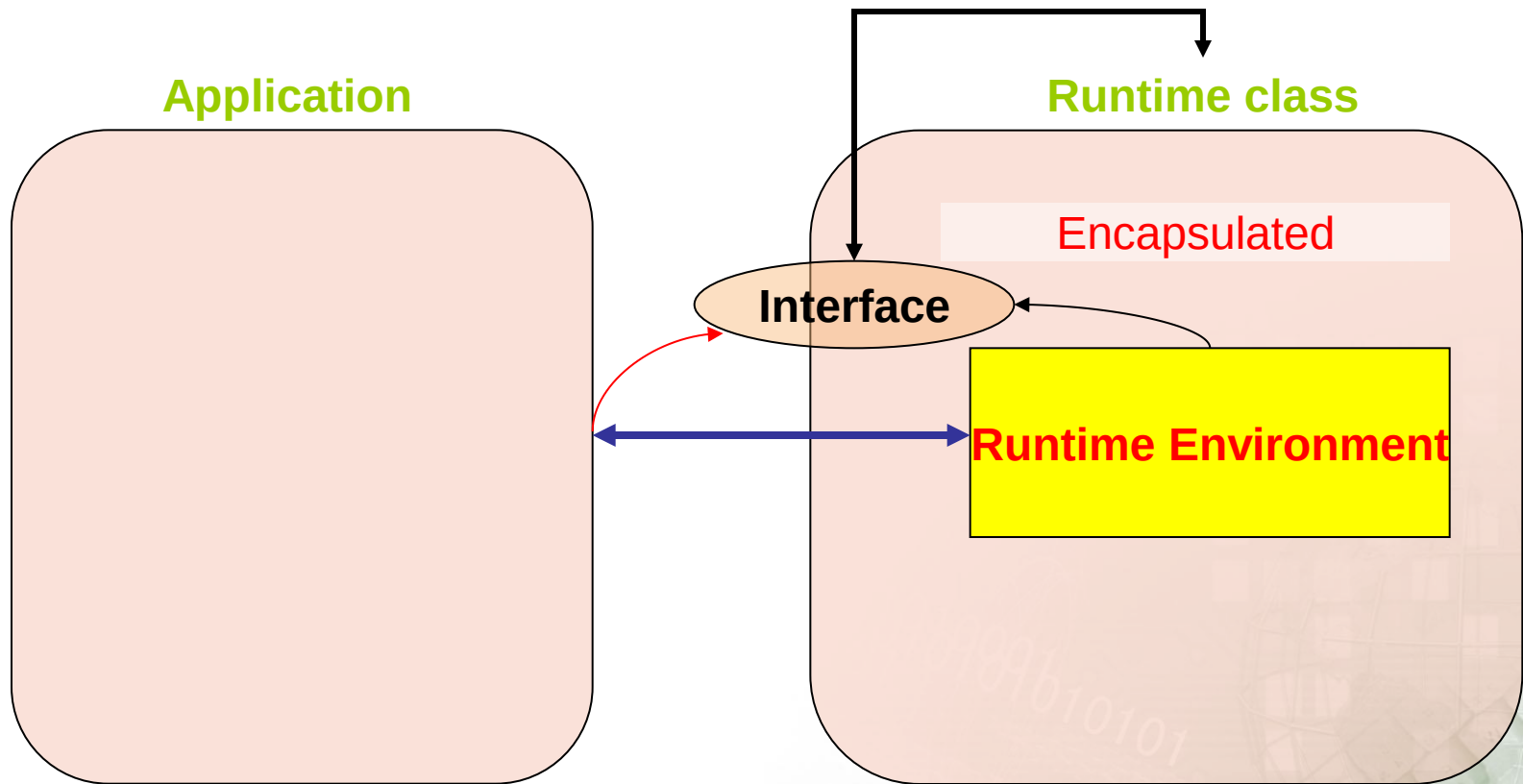
Math class 3-2

Method	Description
<code>double sin (double numvalue)</code>	Calculates sine of angle numvalue
<code>double cos (double numvalue)</code>	Calculates cosine of angle numvalue
<code>double pow (double a, double b)</code>	Calculates a to the power b
<code>double sqrt (double numvalue)</code>	Calculates square root of given value
<code>int abs (int numvalue)</code>	Calculates absolute value of numvalue for int type. Accepts parameters of type float, long and double.
<code>double ceil (double numvalue)</code>	Returns smallest whole number greater than or equal to numvalue.
<code>double floor (double numvalue)</code>	Returns largest whole number less than or equal to numvalue.
<code>int max(int a, int b)</code>	Returns the larger value among int a and int b. It also accepts parameters of type long, float and double.
<code>int min(int a, int b)</code>	Returns the smaller value among a and b. It also accepts parameters of type long, float and double.

Math class 3-3

```
class MathDemo {
    /** Constructor. */
    protected MathDemo() {
    }
    /** main method demonstrates the different methods
of Math class
    * @param args passed to the main method
    */
    public static void main(final String[] args) {
        /** This variable stores the value of num. */
        int num = 38;
        /** This value stores the value of num1. */
        float num1 = 65.7f;
        System.out.println(Math.ceil(num));
        System.out.println(Math.ceil(num1));
        System.out.println(Math.floor(num));
        System.out.println(Math.floor(num1));
        System.out.println(Math.round(num));
        System.out.println(Math.round(num1));
    }
}
```

Runtime class 4-1



Runtime class 4-2

Application

```
...  
Runtime r = Runtime.getRuntime();  
...  
...  
long freemem = r.freeMemory();  
  
long totalmem = r.totalMemory();  
  
r.gc();  
...
```

Runtime class

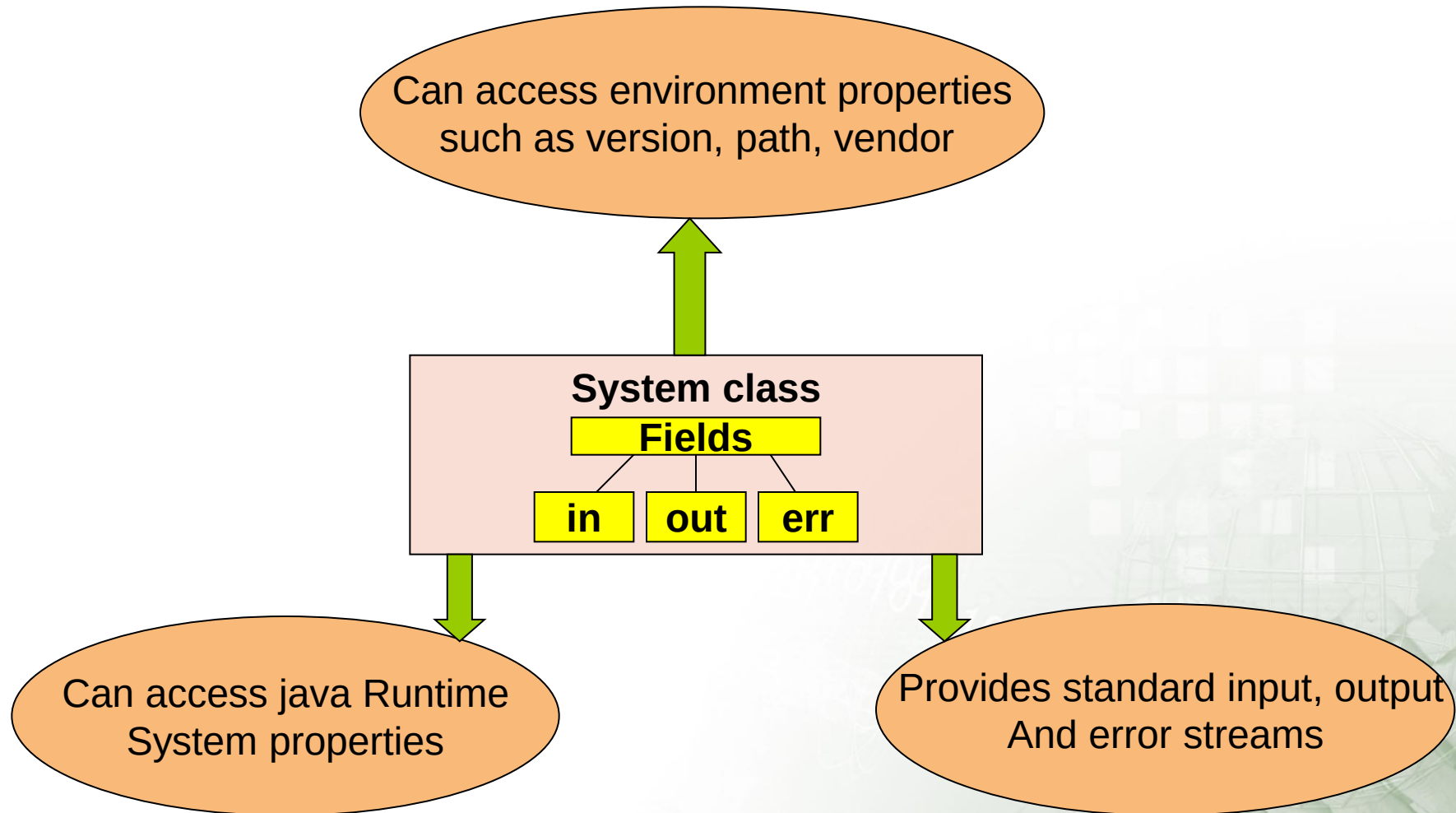
Runtime class 4-3

Method	Description
long freeMemory()	Determines the amount of free memory available to the Java runtime system in terms of bytes.
long totalMemory()	Returns the total amount of memory in the Java virtual machine. It is measured in bytes.
long maxMemory()	Returns the maximum amount of memory Java virtual machine will use. If there is no limit then it will return Long.MAX_VALUE.
Runtime getRuntime()	Returns current runtime instance.
Process exec(String name) throws IOException	Executes a separate program whose name is given.

Runtime class 4-4

```
class RuntimeDemo {
    /** Constructor. */
    protected RuntimeDemo() {
    }
    /** This is a main method.
     * It demonstrates use of Runtime method and
     execution of Calculator program.
     * @param args passed to the main method
     */
    public static void main(final String[] args) {
        Runtime objRun = Runtime.getRuntime();
        Process objProcess = null;
        try {
            objProcess = objRun.exec("calc.exe");
        } catch (Exception e) {
            System.out.println("Error executing Calculator");
        }
    }
}
```

System class 3-1



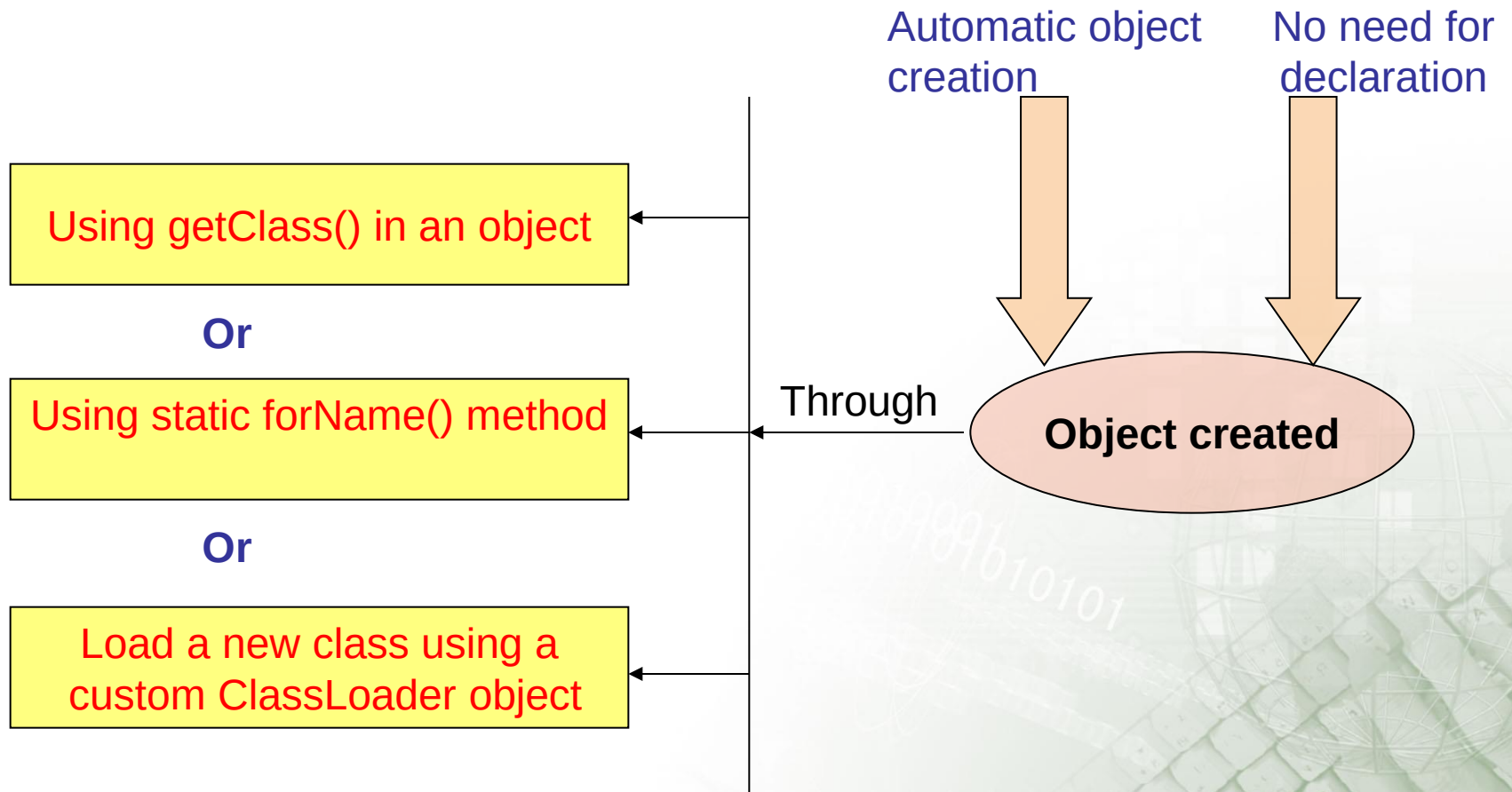
System class 3-2

Method	Description
<code>static void exit(int code)</code>	Halts execution and returns value of code. 0 indicates normal exit.
<code>static Properties getProperties()</code>	Returns properties associated with the Java runtime system.
<code>static void setProperty(Properties p)</code>	Sets the current system properties
<code>static void arrayCopy(Object a, int b, Object c, int d, int size)</code>	Copies an array.

System class 3-3

```
class EnvironmentProperty {
    /** Constructor. */
    protected EnvironmentProperty() {
    }
    public static void main(final String[] args) {
        System.out.println(System.getProperty("java.class.path"));
        System.out.println(System.getProperty("java.home"));
        System.out.println(System.getProperty("java.class.version"));
        System.out.println(System.getProperty("java.specification.vendor"));
        System.out.println(System.getProperty("java.specification.version"));
        System.out.println(System.getProperty("java.vendor"));
        System.out.println(System.getProperty("java.vendor.url"));
        System.out.println(System.getProperty("java.version"));
        System.out.println(System.getProperty("java.vm.name"));
        String []arr1 = {"a", "b", "c"};
        String []arr2 = {"d", "e", "f"};
        System.arraycopy(arr1, 0, arr2, 0, arr1.length);
        int i;
        for (i = 0; i < 3; i++) {
            System.out.println("Array1 position :" + i + " Value: " +
arr1[i]);
            System.out.println("Array2 position :" + i + " Value: " +
arr2[i]);
        }
    }
}
```

“Class” class 2-1



“Class” class 2-2

```
class ClassDemo {
    /** Constructor. */
    protected ClassDemo() {
    }
    /** It demonstrates accessing methods of Class class
     * @param args passed to the main method
     */
    public static void main(final String[] args) {
        StoreString objString = new StoreString();
        StoreInteger objInteger = new StoreInteger();
        Class objClass;
        objClass = objString.getClass();
        System.out.println("objString is object of
type:"+objClass.getName());
        objClass = objInteger.getClass();
        System.out.println("objInteger is object of type: " +
objClass.getName());
        objClass = objClass.getSuperclass();
        System.out.println("objInteger's superclass is " +
objClass.getName());
    }
}
```

Object class in Java

- ❑ The Object class is the parent class of all the classes in java by default.
- ❑ In other words, it is the topmost class of java.
- ❑ The Object class is beneficial if you want to refer any object whose type you don't know.
- ❑ Notice that parent class reference variable can refer the child class object, known as upcasting.
- ❑ **getObject()** method that returns an object but it can be of any type like Employee, Student etc, we can use Object class reference to refer that object.

Object class 2-1

- ❑ Superclass of all classes.
- ❑ By default, user-defined class extends from Object class.

Method	Description
boolean equals(Objectobj)	Compares current object instance with given object and checks if they are equal.
void finalize() throws Throwable	Default form of the finalize method. Usually overridden by subclasses.
String toString()	Returns a String representation of the object.
void wait() throws InterruptedException	Causes current thread to enter waiting state.

Object class 2-2

```
/** This class demonstrates Object class.
 */
class ObjectDemo {
    /** Constructor. */

    protected ObjectDemo() {
        ,
    } else
    System.out.println("Usage: java ObjectDemo Java");
}

public static void main(final String[] args) {
    if(args.length > 0){
        if (args[0].equals("Java")) {
            System.out.println("Yes, Java is the right technology!");
        }else
        System.out.println("Usage: java ObjectDemo Java");
    }
}
```

Autoboxing and Unboxing

- ❑ Autoboxing and Unboxing features was added in Java5.
- ❑ Autoboxing is a process by which primitive type is automatically encapsulated(boxed) into its equivalent type wrapper
- ❑ Auto-Unboxing is a process by which the value of an object is automatically extracted from a type Wrapper class.

Example of Autoboxing and Unboxing

```
class Test {  
public static void main(String[] args) {  
Integer iob = 100; //Auto-boxing of int i.e converting primitive  
                  data type int to a Wrapper class Integer  
int i = iob; //Auto-unboxing of Integer i.e converting Wrapper  
             class Integer to a primitive type int  
System.out.println(i+" "+iob);  
Character cob = 'a'; //Auto-boxing of char i.e converting  
                    primitive data type char to a Wrapper class Character  
char ch = cob; //Auto-unboxing of Character i.e converting  
              Wrapper class Character to a primitive type char  
System.out.println(cob+" "+ch); } }
```

Autoboxing / Unboxing in Expressions

- ❑ Whenever we use object of Wrapper class in an expression, automatic unboxing and boxing is done by JVM.

```
Integer iOb;  
iOb = 100; //Autoboxing of int  
++iOb;
```

- ❑ When we perform increment operation on Integer object, it is first unboxed, then incremented and then again reboxed into Integer type object.
- ❑ **This will happen always, when we will use Wrapper class objects in expressions or conditions etc.**

Benefits of Autoboxing / Unboxing

- ❑ Autoboxing / Unboxing lets us use primitive types and Wrapper class objects interchangeably.
- ❑ We don't have to perform Explicit typecasting.
- ❑ It helps prevent errors, but may lead to unexpected results sometimes. Hence must be used with care.
- ❑ Auto-unboxing also allows you to mix different types of numeric objects in an expression.
- ❑ When the values are unboxed, the standard type conversions can be applied.

Example

```
class Test {  
    public static void main(String args[]) {  
        Integer i = 35;  
        Double d = 33.3;  
        d = d + i;  
        System.out.println("Value of d is " + d); } }
```

- ❑ When the statement `d = d + i;` was executed, `i` was auto-unboxed into `int`, `d` was auto-unboxed into `double`, addition was performed and then finally, auto-boxing of `d` was done into `Double` type Wrapper class.

Summary

- ❑ The classes in a package must be saved under a folder that bears the same name as the package.
- ❑ The `java.lang` package is imported by default into every Java program.
- ❑ Wrapper classes encapsulate simple primitive types in the form of classes.
- ❑ The `StringBuffer` class is used as a building block for building Strings.
- ❑ Strings are immutable, which means they are constant and their value cannot be changed.
- ❑ `Math` is a final class that defines methods for basic numeric operations as well as trigonometric functions.
- ❑ The `Runtime` class encapsulates the runtime environment and is typically used for memory management and running additional programs.